# Lecture 2
# Snapshots: `add`, `commit`, and the Three Areas

Saving versions of your project, the staging area, good messages, and ignoring files

**Learning goals**

- Explain what a *commit* is: a saved snapshot of the whole project.
- Name the three areas (working directory, staging area, repository) and what moves files between them.
- Use `git add` and `git commit` to save versions, and read `git status` at each step.
- Choose which changes go into a commit, and write a clear commit message.
- Tell Git to ignore files with `.gitignore`, and know why secrets must never be committed.

## 1 What is a commit?

In Lecture 1 you created a repository, but it still has *no history*: `git status` said `No commits yet`. History is made of **commits**. A commit is a saved snapshot of your whole project at one moment, together with a message saying what you did, who did it, and when.

A good picture is a save point in a video game. You can keep playing and, if something goes wrong, return to an earlier save point. Unlike a game, you decide when to make a save point and you can name each one.

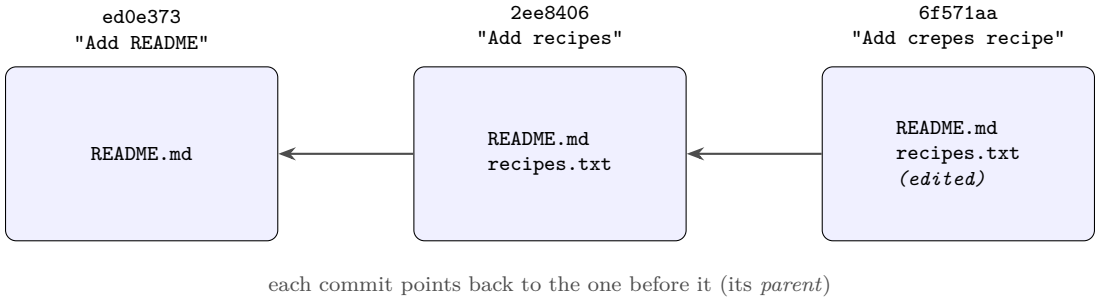



**Figure 1:** A history of three commits. Each box is the state of the whole project saved by that commit. The 7-character code is the commit's ID (yours will differ).

**Key idea**

A commit is a *snapshot* of the whole project, not just a list of what changed. (Git is clever about storage: a file that did not change is not copied again.) You can go back to any commit and see the project exactly as it was then.

**Quick fact**

Every commit has a unique ID, a 40-character code such as `ed0e373a...`. Git usually shows just the first 7 characters. The ID is computed from the commit's contents, so in practice no two commits share one.

## 2 The three areas

To make a commit you do not simply "save". Git uses *three areas*, and understanding them is the most important idea of the whole course.

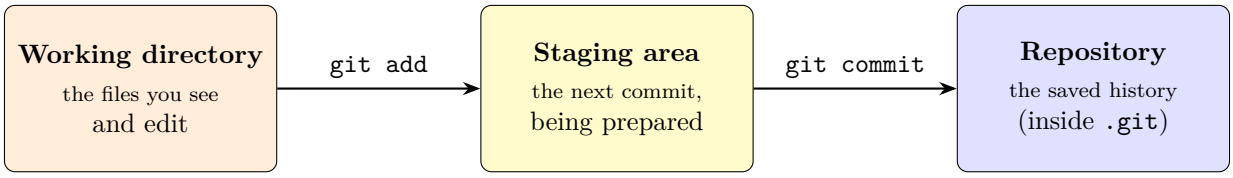



**Figure 2:** The three areas. Changes move from left to right: first you *stage* the changes you want, then you *commit* what is staged.

**Working directory**
The project folder as you see it: the files you create and edit.

**Staging area**
A waiting room for the next commit. You put in it exactly the changes you want to save together. (Git's own name for it is the *index*.)

**Repository**
The repository's saved history: all commits, kept in the hidden `.git` folder.

Why the extra step? Imagine you fixed a typo in one file and also started a half-finished feature in another. The staging area lets you commit *only the typo fix* now, and the feature later, as separate, meaningful commits.

**Quick fact**

The staging area is one of Git's distinctive ideas. Several other version-control tools save all changed files at once, with no step in between.

### 2.1 The life of a file

At any moment, each file in your project is in one of four states:

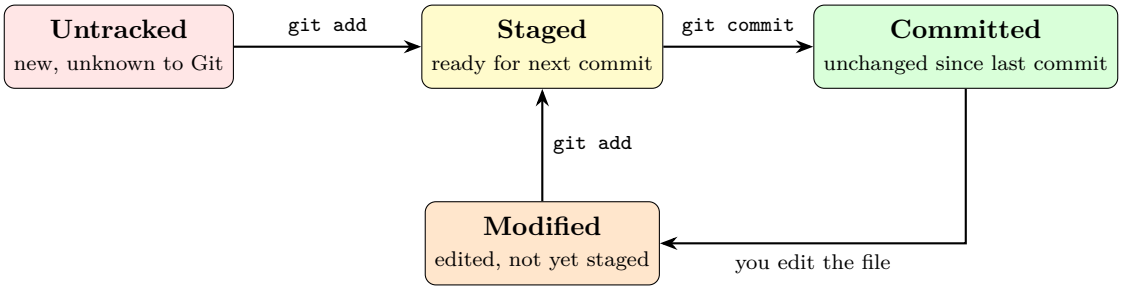



**Figure 3:** States of a file. A file you have never added is *untracked*. After a commit it is *committed*; editing it makes it *modified*; `git add` stages it again.

A file is **tracked** if it is in the latest commit or in the staging area; otherwise it is **untracked**. `git status` reports the states in its own words:

| State | What `git status` prints |
|---|---|
| Untracked | under `Untracked files:` |
| Modified | under `Changes not staged for commit:`, as `modified:` |
| Staged | under `Changes to be committed:` |
| Committed | not listed at all (nothing to report) |

## 3 Your first commit, step by step

We will build a small recipe collection. Make a new folder and a repository, and two files:

```
$ mkdir recipes
$ cd recipes
$ git init
$ echo "# Recipes" > README.md
$ echo "Pancakes: flour, milk, eggs" > recipes.txt
$ git status --short
?? README.md
?? recipes.txt
```

`git status --short` is a compact form of `git status`: one line per file. `??` means untracked. (Committing needs the name and email you set in Lecture 1; if Git answers `Please tell me who you are`, repeat the `git config` step.)

**Step 1: stage.** Put `README.md` in the staging area:

```
$ git add README.md
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   README.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        recipes.txt
```

`README.md` is now *staged* ("Changes to be committed") and `recipes.txt` is still untracked. We chose to stage only one of the two files. (Git's hints in brackets, such as "to unstage", are ways to undo things. We learn them in Lecture 3, so ignore them for now.)

**Step 2: commit.** Save what is staged, with a message after `-m`:

```
$ git commit -m "Add README"
[main (root-commit) ed0e373] Add README
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
$ git status --short
?? recipes.txt
```

The first line says: branch `main`, this is the *root commit* (the very first, with no parent), its ID is

`ed0e373`, and its message. `recipes.txt` is still untracked because we never staged it, so it is *not* part of this commit.

**Step 3: the next commit.** Stage and commit the recipes:

```
$ git add recipes.txt
$ git commit -m "Add recipes"
[main 2ee8406] Add recipes
 1 file changed, 1 insertion(+)
 create mode 100644 recipes.txt
```

**Looking at the history.** The command `git log --oneline` lists your commits, newest first, one per line, as ID and message:

```
$ git log --oneline
2ee8406 Add recipes
ed0e373 Add README
```

These are the first two boxes of Figure 1. (Lecture 3 explores the history in depth. If the output fills the screen and stops, press `q` to quit it.)

**Quick fact**

On Windows, Git may print a warning such as `LF will be replaced by CRLF` when you `git add`. It is about line endings, is harmless here, and can be ignored.

**Pitfall**

Committing does nothing with a file you never staged. A very common beginner surprise is a missing file in the commit because it was untracked. Run `git status` *before* committing and read it.

## 3.1 Modified files and selective commits

Now edit both files. Both are tracked now, so they show as *modified*:

```
$ echo "Crepes: flour, milk, eggs" >> recipes.txt
$ echo "- add more recipes" >> README.md
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md
        modified:   recipes.txt

no changes added to commit (use "git add" and/or "git commit -a")
$ git status --short
 M README.md
 M recipes.txt
```

In the short form each line has *two* columns before the file name: the **left** column describes the staging area, the **right** column the working directory. So `M` (a space, then `M`) means "modified in the working directory, not staged". Now stage only one file and look again:

```
$ git add recipes.txt
$ git status --short
 M README.md
```

```
M  recipes.txt
```

`M` (`M` in the left column) means "staged". `recipes.txt` is in the staging area and `README.md` is not, so a commit now includes `recipes.txt` only. Commit it, then stage and commit the other file as its own commit:

```
$ git commit -m "Add crepes recipe"
$ git add README.md
$ git commit -m "Update README"
$ git status
On branch main
nothing to commit, working tree clean
```

`working tree clean` means nothing is modified, staged or untracked. This table follows both files through these steps:

| After this step | `README.md` | `recipes.txt` |
|---|---|---|
| both edited | modified | modified |
| `git add recipes.txt` | modified | **staged** |
| `git commit -m "Add crepes recipe"` | modified | committed |
| `git add README.md` | **staged** | committed |
| `git commit -m "Update README"` | committed | committed |

The same file can even be in two areas at once: if you stage a file and then edit it again, the *staged* version is the earlier one and the later edit is still "modified". `git status` then lists the file under both headings (`MM` in the short form).

**Try it**

Make a new repository, then create and commit two files. Edit both. Stage only one, run `git status` and `git status --short`, then commit. Which file is left over, and in which state? Then stage and commit the other one, and look at `git log --oneline`.

## 4 Handy variations

| Command | Meaning |
|---|---|
| `git add file1 file2` | Stage several named files. |
| `git add .` | Stage *everything* new or changed in the current folder and below. Convenient, but check `git status` first so you do not stage what you did not mean to. |
| `git commit -m "msg"` | Commit what is staged, with a one-line message. |
| `git commit -am "msg"` | Shortcut: stage all changes to files Git already tracks, then commit. It does *not* include untracked (new) files. |

Deleting files, undoing a staged change and going back to an older version are the subject of Lecture 3.

## 5 Writing good commit messages

The message is for your future self and your teammates. Months later, `Fixed stuff` tells nobody anything. A common convention:

- A short first line (about 50 characters), written as a command: *Add*, *Fix*, *Remove*, *Update*.
- It says *what* the commit does, and, when it matters, why.
- No full stop at the end.

| Weak | Better |
|---|---|
| `stuff` | `Add pancake recipe` |
| `fixed it` | `Fix typo in README title` |
| `update` | `Update serving size for crepes` |
| `asdf` | `Remove unused test file` |

**Pitfall**

If you type `git commit` *without* `-m`, Git opens a text editor for the message. On many computers the default editor is `vim`, which confuses beginners (it does not accept normal typing). To get out without committing, press `Esc`, type `:q!` and press Enter. To avoid the problem, either always use `-m "..."`, or choose an editor you know, for example `git config --global core.editor "nano"` (if `nano` is available on your computer).

**Key idea**

Make *small, focused* commits: one idea per commit. They are easier to understand, to review, and (in later lectures) to undo.

## 6 Ignoring files: `.gitignore`

Some files should never be in the history: generated files, temporary logs, large downloads, and above all *secrets* such as passwords. Tell Git to ignore them with a plain text file named `.gitignore` in the project folder:

```
# generated and private files
*.log
secret.env
build/
```

Each line is a *pattern*:

- `*.log` ignores every file whose name ends in `.log` (`*` matches any characters within a name);
- `secret.env` ignores that exact file;
- `build/` ignores any folder named `build` (the trailing `/` means folder);
- a line starting with `!` is an exception placed *after* the rule it overrides (`!keep.log` un-ignores that file);
- a line starting with `#` is a comment. Comments must be on their own line.

Continue in the clean `recipes` repository. Create a log file and the ignore file (the first `echo` uses `>` to start the file, the others use `>>` to append; a text editor works too):

```
$ echo "temporary output" > debug.log
$ echo "# generated and private files" > .gitignore
$ echo "*.log" >> .gitignore
$ echo "secret.env" >> .gitignore
$ echo "build/" >> .gitignore
```

```
$ git status --short
?? .gitignore
```

`debug.log` does not appear: Git ignores it. The `.gitignore` file itself is untracked, and *should* be committed so that everyone who clones the project ignores the same things:

```
$ git add .gitignore
$ git commit -m "Add .gitignore"
```

Your history now has five commits, and `git log --oneline` shows them all.

**Pitfall**

**Ignoring only affects files that are not yet tracked.** If you already committed a file, adding it to `.gitignore` does not remove it from the project or from the history. (Lecture 3 shows how to stop tracking a file.)

**Pitfall**

**Never commit passwords, keys or tokens.** Git remembers every commit, so deleting the file later does *not* erase it from history. If a secret is ever committed, treat it as leaked: change the password or revoke the key. Put such files in `.gitignore` *before* the first `git add`.

# 7 Homework 2

**Homework 2: Snapshots**

From the course folder (`LearningGit/`) run:

```
$ bash codeHints/L02/setup_hw2.sh
$ cd practice/hw2
```

The setup script creates a repository `practice/hw2` (with a local identity already set) containing five files and a folder, and **no commits**: `README.md`, `notes.txt`, `todo.txt`, `debug.log`, `secret.env`, and the folder `build/` (which contains one file, `output.bin`). Run `git status` to look. Then do the tasks *in this order*:

**2.1 First commit**
Commit `README.md` and `notes.txt` *only*, with the message `Add README and notes`.

**2.2 Second commit**
Commit `todo.txt` with the message `Add todo list`.

**2.3 Ignore**
Create a `.gitignore` so that `debug.log`, `secret.env` and the `build/` folder are ignored. Commit it with the message `Add .gitignore`. After this, `git status` must not list the three ignored items, and `git log --oneline` shows three commits.

**2.4 Two areas at once**
Append one new line to `notes.txt` and one new line to `todo.txt` (use `echo "..." >> file`). Stage `notes.txt` *only*, and do **not** commit (the checker expects the changes to be uncommitted). Then `git status --short` must show exactly two lines: `M  notes.txt` (staged) and ` M todo.txt` (modified, not staged).

**2.5 Question**
Outside the repository, in the file `practice/hw2_areas.txt`, write the name of the area that `git add` copies changes into.

**Check yourself:** from the course folder, `bash codeHints/L02/check_hw2.sh`. All `PASS`, then compare with the solutions at the end of this lecture (or in `solutions/L02/`).

## 8 Summary

- A commit is a snapshot of the whole project plus a message, author and date. History is a chain of commits, each pointing to its parent.
- Three areas: working directory → (`git add`) staging area → (`git commit`) repository.
- Files are untracked, modified, staged or committed; `git status` tells you which.
- Stage only what belongs together; write short, imperative messages.
- `.gitignore` keeps unwanted files out. Never commit secrets.

**Further reading:** *Pro Git* (https://git-scm.com/book), Chapter 2.2 "Recording Changes to the Repository" (it also covers `.gitignore`); manual pages `git help add`, `commit`, `gitignore`.

**Solutions**

Stop here if you have not tried the homework yet.
Attempt every task first, run the checker, and only then read on.

## Solutions to Homework 2

Run from the course folder, then work inside `practice/hw2`:

```
$ bash codeHints/L02/setup_hw2.sh
$ cd practice/hw2
$ git add README.md notes.txt
$ git commit -m "Add README and notes"
$ git add todo.txt
$ git commit -m "Add todo list"
$ echo "debug.log" > .gitignore
$ echo "secret.env" >> .gitignore
$ echo "build/" >> .gitignore
$ git add .gitignore
$ git commit -m "Add .gitignore"
$ echo "Second note." >> notes.txt
$ echo "- practice commits" >> todo.txt
$ git add notes.txt
$ git status --short
M  notes.txt
 M todo.txt
$ echo "staging area" > ../hw2_areas.txt
```

**2.1.** Staging only the two named files leaves `todo.txt` untracked, so it is not in the first commit. `git add .` here would have wrongly included `todo.txt`, `debug.log`, `secret.env` and `build/`. If you made a mistake before committing, `git status` shows what is staged; you will learn to unstage in Lecture 3.

**2.2.** Commit `todo.txt` on its own, so the history reads naturally.

**2.3.** Create the ignore file *before* anything adds the secrets. Check that `git status` no longer mentions the three items. (Optional: `git check-ignore -v debug.log secret.env build/output.bin` prints which rule matched each file.) `*.log` instead of `debug.log` also passes the checker.

**2.4.** The *first* column of `git status --short` describes the staging area, the *second* the working directory. `M  notes.txt` means "staged change"; ` M todo.txt` means "modified but not staged". A commit now would include only `notes.txt`. (Do not commit: the checker expects this state.)

**2.5.** The **staging area**, also called the **index**. The answer file sits in `practice/`, outside `hw2`, so that it does not appear as an untracked file in the repository.

**Common mistakes.**

- Using `git add .` too early, which stages `secret.env` and the build files. Look at `git status` first.
- Wrong commit messages: the checker compares them exactly, including capital letters.
- Committing in task 2.4. If you did, the checker will report four commits. You can run `setup_hw2.sh --reset` and do the tasks again.
- Forgetting `-m`: an editor opens. Press `Esc`, type `:q!`, press Enter, and try again.